Software-Based IEEE 754 Floating Point Converter

1. Abstract

This project involves the design and implementation of a software-based simulator to convert decimal numbers into the IEEE 754 Single Precision (32-bit) floating-point format. Developed using **Scilab**, the simulator automates the normalization, biasing, and bit-assembly processes, providing a clear visualization of how computers store fractional data in memory.

2. Introduction

In computer architecture, the **IEEE 754 Standard** is the technical benchmark for floating-point computation. It is vital for maintaining numerical consistency across different hardware platforms. For students in AI and Data Science, understanding this format is critical because it dictates the precision of model weights and the efficiency of mathematical operations in GPUs.

3. Technical Specifications

The simulator implements the **Single Precision** format, which consists of 32 bits divided into three fields:

1. **Sign Bit (1 bit):** Determines if the number is positive (0) or negative (1).
2. **Biased Exponent (8 bits):** Uses an excess-127 representation to handle both large and small magnitudes.
3. **Mantissa/Significand (23 bits):** Represents the precision of the number using a "hidden bit" (Implicit 1) strategy to maximize memory efficiency.

4. Methodology & Algorithm

The software follows a 4-step logical flow to achieve the conversion:

- **Step 1: Decomposition:** The input decimal is split into its integer and fractional parts.
- **Step 2: Binary Conversion:** * Integer part is converted via repeated division by 2.
    - Fractional part is converted via repeated multiplication by 2.
- **Step 3: Normalization:** The binary string is shifted until only one '1' remains before the binary point ($1.M \times 2^E$).
- **Step 4: Biasing:** The exponent (E) is added to the bias constant (127).

- **Step 5: Concatenation:** The sign, 8-bit exponent, and 23-bit mantissa are joined to form the final 32-bit string.

5. Implementation Details

The project is implemented in **Scilab**, utilizing its robust string handling and mathematical functions. Unlike hardware-based converters using logic gates, this software-based emulator provides a high-level logic overview that is easily modifiable for different precisions (like 64-bit Double Precision).

Key Scilab Functions Used:

- dec2bin(): To handle integer conversions.
- while loops: To calculate the fractional binary string.
- strcat(): To assemble the final 32-bit bitstream.

6. Results & Discussion

The simulator was tested with several cases:

- **Case 1 (Positive Fraction):** $13.125 \rightarrow$ Result: 0 10000010 10100100000000000000000
- **Case 2 (Negative Fraction):** $-0.75 \rightarrow$ Result: 1 01111110 10000000000000000000000

The outputs were verified against standard IEEE 754 tables, confirming 100% accuracy in bit placement and biasing.

7. Conclusion

This project successfully demonstrates the internal mechanics of floating-point representation. By building a software-based emulator rather than a physical circuit, we achieved a deeper understanding of the bit-level logic that powers modern AI frameworks and data processing engines.